

Module 2: Data Analysis and Visualization

Data Summarization: One Quantitative and Categorical Variable. Data Classes: One Dimensional Data Classes-Data Frames and Matrices-Lists. Data Cleaning: Dealing with Missing Data-Strings and Recoding Variables. Manipulating Data in R: Reshaping Data-Merging Datasets. Data Visualizations: Plotting with ggplot2- Plotting with Base R



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Data Analysis and Visualization

- Data Summarization: One Quantitative and Categorical Variable.
- Data Classes: One Dimensional Data Classes-Data Frames and Matrices-Lists.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Data Summarization

- In any statistical analysis, **data** is defined as a collection of information, which may be used to prove or disprove a hypothesis or data set.
- Although data can take on any form, however, it's classified into two main categories depending on its nature—**categorical and numerical data**.
- Understanding different data types helps you to choose which method is best for any situation.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Key Takeaways:

- Quantitative or numerical data and categorical data are both incredibly important for statistical analysis.
- There are different types of both data that can result in unique (and very useful) data analysis results.
- Data collection methods are easier to conduct than you may think.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Population, Sample, and Observations

- The units on which we measure data—such as persons, cars, animals, or plants— are called observations.
- These units/observations are represented by the Greek symbol ω .
- The collection of all units is called population and is represented by Ω .
- When we refer to $\omega \in \Omega$, we mean a single unit out of all units, e.g. one person out of all persons of interest.
- If we consider a selection of observations $\omega_1, \omega_2, \dots, \omega_n$, then these observations are called sample. A sample is always a subset of the population, $\{\omega_1, \omega_2, \dots, \omega_n\} \subseteq \Omega$.
- Example: If we are interested in the social conditions under which Indian people live, then we would define all inhabitants of India as Ω and each of its inhabitants as ω . If we want to collect data from a few inhabitants, then those would represent a sample from the total population.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Variables

- If we have specified the population of interest for a specific research question, we can think of what is of interest about our observations.
- A particular feature of these observations can be collected in a statistical variable X . Any information we are interested in may be captured in such a variable.
- For example, if our observations refer to human beings, X may describe marital status, gender, age, or anything else which may relate to a person.

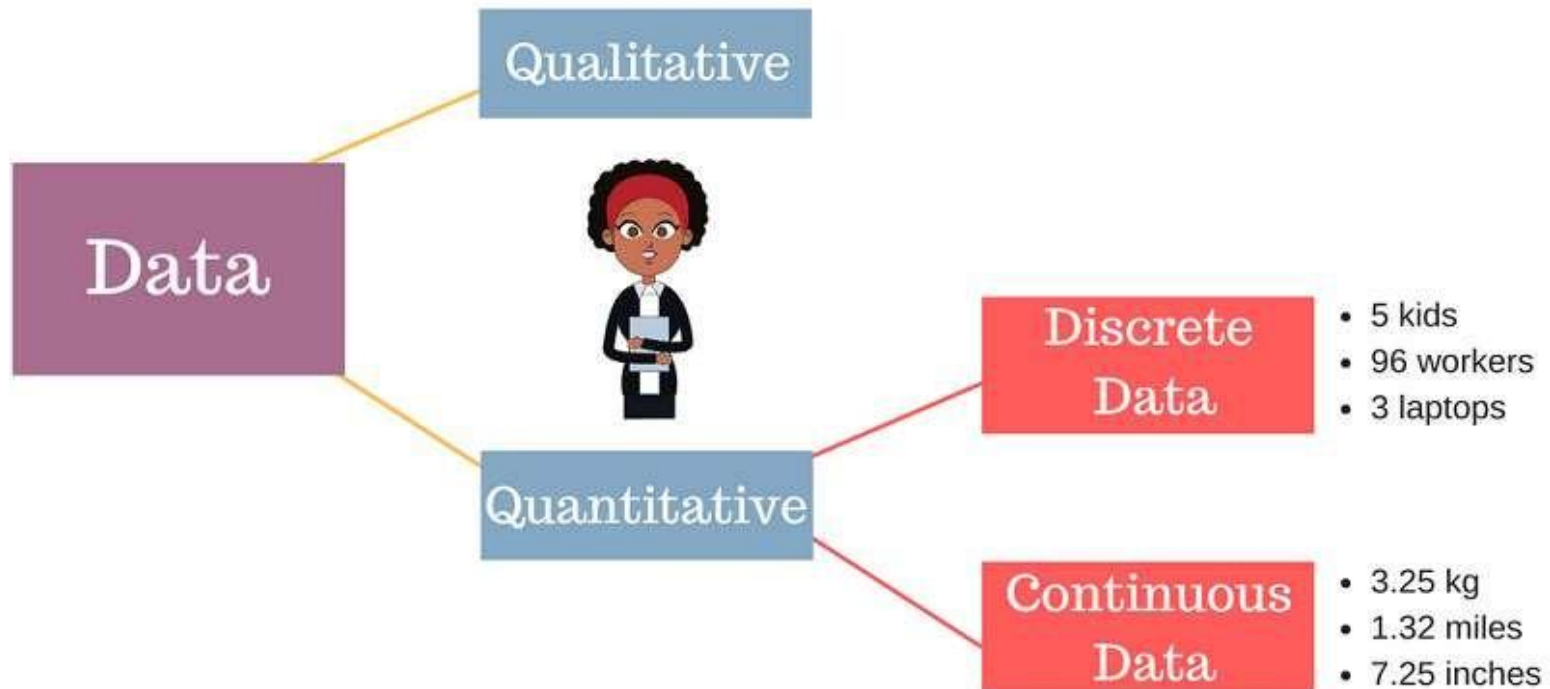


**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Contd



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Contd

- Of course, we can be interested in many different features, each of them collected in a different variable X_i , $i = 1, 2, \dots, p$.
- Each observation ω takes a particular value for X . If X refers to gender, each observation, i.e. each person, has a particular value x which refers to either “male” or “female”.
- The formal definition of a variable is

$$X : \Omega \rightarrow S$$

$$\omega \rightarrow x$$

- This definition states that a variable X takes a value x for each observation $\omega \in \Omega$, whereby the number of possible values is contained in the set S



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Example of Variable

- If X refers to gender, possible x -values are contained in $S = \{\text{male, female}\}$. Each observation ω is either male or female, and this information is summarized in X .
- Let X be the country of origin for a car. Possible values to be taken by an observation ω (i.e. a car) are $S = \{\text{Italy, South Korea, Germany, France, India, China, Japan, USA, ...}\}$.
- A variable X which refers to age may take any value between 1 and 125. Each person ω is assigned a value x which represents the age of this person.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Qualitative Variable

- Qualitative variables are the variables which take values x that cannot be ordered in a logical or natural way.
- For example,
 1. the colour of the eye
 2. the name of a political party
 3. the type of transport used to travel to work



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Quantitative Variable

- Quantitative variables represent measurable quantities.
- The values which these variables can take can be ordered in a logical and natural way.
- Examples of quantitative variables are
 1. size of shoes,
 2. price for houses,
 3. number of semesters studied
 4. weight of a person



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Qualitative Data



VS

Quantitative Data



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Quantitative Data

- Countable or measurable
- Relates to numbers
- Factual i.e. fixed values
- Source - counting or measurement
- Data can be measured accurately
- Structured data collection methods

Qualitative Data

- Descriptive
- Relates to words or language
- Subjective i.e. open to interpretation
- Source - observations, interviews, open dataset
- Data can be observed, not measured
- Semi-structured or unstructured data collection methods



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Discrete and Continuous Variables

Discrete Variables:

- Variables which can only take a finite number of values.
- Industrial sector variable values are telecommunication industry, retail industry with finite number of possible values.
- **All qualitative variables are discrete, like:**
 1. the colour of the eye or
 2. the region of a country
- **Quantitative variables can be discrete:**
 1. the number of semesters studied would be discrete because the number of values these variables can take is limited.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Continuous Variable

- Variables which can take an infinite number of values are called continuous variables.
- Can take any numeric value within a range

Examples are:

1. the time it takes to travel to university,
2. the length of an antelope, and
3. the distance between two planets. Sometimes, it is said that continuous variables are variables which are “measured rather than counted”.
4. Patient's weight (e.g., 153.2 lb, 98.2 lb)



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Data Classes: One Dimensional Data Classes

- A data class is a list of data set allocation attributes and their values.
- Basic data structures are **vectors, matrices, arrays, lists, and data frames**. They can contain numeric values, logical values or even characters (strings).

❖ **Data Frames**

❖ **Matrices**

❖ **Lists**

❖ **Arrays**

❖ **Vector**



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



❖ Data Frames

- Data frames are used to store tabular data in R.
- They are an important type of object in R and are used in a variety of statistical modeling applications.
- Most data sets are stored in R as data frames. They are like matrices, but with the columns having their own names.
- Unlike matrices, data frames can store different classes of objects in each column. Matrices must have every element be the same class.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Example:

The data frame *women* contains the average weights (in pounds) of American women aged 30 to 39 of particular heights (in inches):

```
women
```

	height	weight
1	58	115
2	59	117
3	60	120
4	61	123
5	62	126
6	63	129
7	64	132
8	65	135
9	66	139
10	67	142
11	68	146
12	69	150
13	70	154
14	71	159
15	72	164



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Sometimes, the data frames can be very large, it is not a good idea to display the entire data set. To obtain the basic information of the data set, we can use

```
nrow(women) #show the number of rows
```

```
[1] 15
```

```
ncol(women) #show the number of columns
```

```
[1] 2
```

```
dim(women) #show both dimensions
```

```
[1] 15 2
```

```
rownames(women) #show the row names
```

```
[1] "1" "2" "3" "4" "5" "6" "7" "8" "9" "10" "11" "12" "13"  
[15] "15"
```

```
colnames(women) #show the column names
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Accessing Subgroups in Data Frames

- Subgroups , i.e. groups of observations which share the same feature(s) of one or several variables, can be accessed using the subset command.
- `subset(painters , School=="F")`
- Accesses all painters for which School=="F" holds.

	Composition	Drawing	Colour	Expression	School
Durer	8	10	10	8	F
Holbein	9	10	16	13	F
Pourbus	4	15	6	6	F
VanLeyden	8	6	6	4	F



Constructing Matrix objects

A **matrix** is a rectangular way of storing data. You can simply view it as a way to store data. Matrices can be constructed using the functions **matrix()**.

```
m<-matrix(1:6,nrow=2,ncol=3)
m
```

	[,1]	[,2]	[,3]
[1,]	1	3	5
[2,]	2	4	6

Matrices are constructed column-wise, so entries can be thought of starting in the “upper left” corner and running down the columns. If you need the matrix to be filled by rows, then add the **byrow=TRUE** argument.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



We can then access elements using two indices. For example,

```
m[1,2]
```

```
[1] 3
```

```
m[1,2:3]
```

```
[1] 3 5
```

Whole rows or columns of matrices may be selected by leaving one index blank:

```
m[1,] #display the first row of m
```

```
[1] 1 3 5
```

```
m[,1] #display the first column of m
```

```
[1] 1 2
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Matrices can also be created by **cbind()**, or **rbind()**.

```
x<-1:3  
y<-10:12  
z<-5:-3  
cbind(x,y,z)
```

```
      x  y  z  
[1,] 1 10 -5  
[2,] 2 11 -4  
[3,] 3 12 -3
```

```
rbind(x,y,z)
```

```
      [,1] [,2] [,3]  
x         1     2     3  
y        10    11    12  
z        -5    -4    -3
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



❖ Lists

- A list can contain objects of different types.
- For example, a list element can be a vector or matrix.
- Lists can be initialized by the command list and can grow dynamically.
- It is important to understand that list elements should be accessed by the name of the entry via the dollar sign or using double brackets



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013




```
x <- list(one=c(1,2,3,4,5),two=c("Hello", "world", "!"))  
x  
$one  
[1] 1 2 3 4 5  
  
$two  
[1] "Hello" "world" "!"  
  
x[[2]]  
[1] "Hello" "world" "!"  
  
x$one  
[1] 1 2 3 4 5
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



❖ Arrays

- Arrays are more general data structures than vectors and matrices in the sense that they can have more than two dimensions. For instance,

```
x <- array(data=1:12, dim=c(3,2,2))
```
- The above example creates a three-dimensional array with
 $3 \cdot 2 \cdot 2 = 12$ elements.
- While matrices are confined to two dimensions, arrays can be of any number of dimensions.
- The array function takes a dim attribute which creates the required number of dimension.



Contd

- In the below example we create an array with two elements which are 3x3 matrices each.
- **# Create an array**

```
a <- array(c('green', 'yellow'), dim=c(3,3,2)) print(a)
```

- When we execute the above code, it produces the following result:

```
, , 1
[,1] [,2] [,3]
[1,] "green" "yellow" "green"
[2,] "yellow" "green" "yellow"
[3,] "green" "yellow" "green"
, , 2
[,1] [,2] [,3]
[1,] "yellow" "green" "yellow"
[2,] "green" "yellow" "green"
[3,] "yellow" "green" "yellow"
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



❖ Vector

1. A **numeric vector** of length 5 can be constructed by the command

```
x <- vector(mode="numeric", length=5)
```

The elements can be accessed by squared brackets: [].

For example, `x[3] <- 4`

2. **Logical vectors** containing the values TRUE and FALSE

1 refers to TRUE

0 refers to FALSE

Example `x.log <- vector(mode="logical", length=4)`

`x.log[1] = x.log[3] = x.log[4] = TRUE`

`mean(x.log)` **Output:-** [1] 0.75

Because the mean of (1,0,1,1)=(TRUE,FALSE,TRUE, TRUE) is 0.75



Indexing

- The standard ways of accessing / indexing elements in a vector, matrix, list, or data frame have discussed, but R allows a lot more flexible accessing of elements.

1. Selecting elements using vectors of positive numbers (letters and LETTERS show the 26 letters of the alphabet)

```
letters[1:3]
```

```
letters[ c(2,4,6) ]
```

```
[1] "a" "b" "c"
```

```
[1] "b" "d" "f"
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



2. Selecting elements using logical vectors

```
x <- 1:10           # numbers 1 to 10
x[ (x>5) ]          # selecting any number >5
x[ (x%%2==0) ]      # numbers that are divisible by 2
x[(x%%2==1)]        # numbers that are not divisible by 2
x[5] <- NA          # 5th element of x is defined
                    # to be missing (NA)

x
y <- x[!is.na(x)]   # all x which are not missing
y
```

returns the output

```
[1]  6  7  8  9 10
[1]  2  4  6  8 10
[1]  1  3  5  7  9
[1]  1  2  3  4 NA  6  7  8  9 10
[1]  1  2  3  4  6  7  8  9 10
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



3. Selecting (deleting) elements using negative numbers

```
x <- 1:10
```

```
x[-(1:5)] # x, but delete first five entries of x
```

returns the output

```
[1] 6 7 8 9 10
```

because the first five elements have been removed.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



4. Selecting elements using characters

```
x <- c(Water=1, Juice=2, Lemonade=3 )  
names(x)  
x["Juice"]
```

returns the output

```
[1] "Water"    "Juice"    "Lemonade"  
Juice  
2
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Loops and Conditions

The concept of loops is convenient when some operation has to be repeated. Loops can be utilized in various ways, for example, via `for` or `while`. Conditions are specified with the `if` statement. For example,

```
x <- 1:10
for(i in 1:10){ if(x[i]>5){x[i] <- x[i]+i}
}
x
```

returns

```
[1] 1 2 3 4 5 12 14 16 18 20
```

In this example, `x` is a vector with 10 elements: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10. For each element `x[i]`, we replace `x[i]` with `x[i]+i` if the condition `x[i]>5` is true; otherwise we do not.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Data Cleaning

- Data cleaning refers to the process of detecting and fixing errors, inconsistencies, and inaccuracies in a dataset to enhance its overall quality and reliability.
- This process typically involves addressing missing data, correcting entry mistakes, standardizing data formats, eliminating duplicate records, and resolving any discrepancies.
- The goal is to ensure that the dataset is accurate, complete, and properly structured for effective analysis or modeling.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Dealing with Missing Data

- Missing values indicate that data is absent or not recorded for a particular entry in the dataset.
- They are not same as zero or an empty string; they explicitly signify that no data is available for that specific cell.
- In R, missing values are represented by the special value NA, which stands for "Not Available" or "Not Applicable."



Dealing with Missing Data

- One of the most important aspects of data cleaning is missing values. Some useful functions for checking your data include :
 - `is.na` - is `TRUE` if the data is `NA` `FALSE` otherwise
 - `!` - negation (NOT)
 - if `is.na(x)` is `TRUE` , then `!is.na(x)` is `FALSE`
 - `all` takes in a `logical` and will be `TRUE` if ALL are `TRUE`
 - `all(!is.na(x))` - are all values of `x` NOT `NA`
 - `any` will be `TRUE` if ANY are true
 - `any(is.na(x))` - do we have any `NA` 's in `x` ?
 - `complete.cases` - returns `TRUE` if EVERY value of a row is NOT `NA`
 - very stringent condition
 - `FALSE` missing one value (even if not important)



Types of “missing” data in R include:

- `NA` - general missing data
- `NaN` - stands for “**N**ot **a** **N**umber”, happens when you do $0/0$.
- `Inf` and `-Inf` - Infinity, happens when you divide a positive number (or negative number) by 0.

We need to be able to check for these R missing types in our data. Each missing data type has a function that returns `TRUE` if the data is missing:

- `NA` - `is.na`
- `NaN` - `is.nan`
- `Inf` and `-Inf` - `is.infinite`
- `is.finite` returns `FALSE` for all missing data and `TRUE` for non-missing



• Common Causes of Missing Data

- **Human Error:** Typos or missed entries during data collection.
- **Incomplete Responses:** Participants skipping questions in surveys or forms.
- **Data Corruption:** Loss or corruption of data during transmission or storage.
- **Inconsistent Data Collection:** Different methods or instruments used to collect data may result in missing information.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



- There are several types of missing data:
- 1. Missing Completely at Random (MCAR):** The missingness is completely random and unrelated to both observed and unobserved data.
 - For example, if data is missing due to a random malfunction in data collection equipment, it's MCAR.
 - 2. Missing at Random (MAR):** The missingness is related to the observed data but not to the missing data itself.
 - For example, if people with lower income levels are more likely to skip questions about income, but the likelihood of missing data is unrelated to their income levels, it's MAR.



3. Missing Not at Random (MNAR): The missingness is related to the value of the missing data itself.

- For instance, if individuals with higher incomes are more likely to skip questions about their income, the missing data is MNAR



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Identify the missing data in a dataset using functions

- Here are the commonly used functions to identify missing data:
 1. **is.na()**: This function checks for missing values (NA) in a dataset and returns a logical matrix (TRUE for missing values and FALSE otherwise).
 2. **sum(is.na())**: You can use sum() with is.na() to count the total number of missing values in a dataset.



3. **anyNA()**: This function returns TRUE if any missing value is present in the dataset or FALSE if none are missing.
4. **complete.cases()**: This function identifies complete cases (rows with no missing values) and returns a logical vector (TRUE for rows without missing values and FALSE for rows with missing values).
5. **colSums(is.na())** and **rowSums(is.na())**: These functions allow you to compute the number of missing values per column or row in the dataset.



6. Using `sapply()`: to Detect Missing Data by
Columns `sapply()` can be used to apply `sum(is.na())`
to each column and provide a column-wise
summary of missing values.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



- One important aspect (esp with subsetting) is that logical operations return NA for NA values. Think about it, the data could be > 2 or not. We don't know, so R says there is no TRUE or FALSE, so that is missing

```
x = c(0, NA, 2, 3, 4, -0.5, 0.2)
x > 2
```

```
[1] FALSE    NA FALSE   TRUE   TRUE FALSE FALSE
```

So what do we do to handle the NA ? What if we want if `x > 2` and `x` isn't NA ?
Don't do `x != NA`, do `x > 2` and `x` is NOT NA :

```
x != NA # WRONG!
```

```
[1] NA NA NA NA NA NA NA
```

```
x > 2 & !is.na(x)
```

```
[1] FALSE FALSE FALSE   TRUE   TRUE FALSE FALSE
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Review the missing data in a dataset using functions

Function	Purpose
<code>is.na()</code>	Identifies missing values (returns TRUE/FALSE)
<code>sum(is.na(data))</code>	Total count of missing values in the dataset
<code>colSums(is.na(data))</code>	Missing values per column
<code>rowSums(is.na(data))</code>	Missing values per row
<code>complete.cases()</code>	Identifies rows with no missing values
<code>aggr() (VIM)</code>	Visualize missing data patterns
<code>md.pattern() (mice)</code>	Shows a matrix of missing data combinations



Apply techniques to handle the missing values

- Here are several techniques in R to handle missing values using appropriate functions.

1. Remove Missing Values

You can remove **rows** with missing values using the `na.omit()` or `drop_na()` (from the `tidyr` package) functions.

Remove Columns with Many Missing Values: If a column has a high percentage of missing values, then we can drop the column.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



2.Impute Missing Values

- **Mean Imputation:** Replace missing values with the mean of the corresponding column.
- **Median Imputation:** Replace missing values with the median of the column, which is less sensitive to outliers.
- **Mode Imputation:** For categorical variables, replace missing values with the most frequent category (mode).



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



3. Predictive Imputation

- K-Nearest Neighbors (KNN) Imputation: Impute missing values based on the nearest neighbors using the DMwR package.
- Multiple Imputation: Impute missing values multiple times using the mice package. This is useful for more complex imputation based on multiple variables.
- **Interpolation (For Time Series Data):** For time series data, missing values can be filled using interpolation techniques like linear or spline interpolation.



Recoding Variables

- Recoding is the process of transforming or reclassifying the values of a variable into new values or categories.
- This is commonly used in data cleaning and preparation to make variables more meaningful or suitable for analysis.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



- **Purpose of Recoding**
- **Simplify Data:** Recoding can reduce the complexity of a variable, making it easier to analyze or interpret.
 - For example, collapsing a multi-level categorical variable into fewer levels.
- **Prepare Data for Analysis:** Recoding may be necessary to meet the requirements of certain statistical models or visualizations.
- **Create New Insights:** Grouping values or transforming continuous variables into categorical bins can uncover new patterns in the data.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



- **Use Cases of Recoding Collapsing**
- **Categorical Variables:** Converting multiple categories into a smaller set of meaningful categories (e.g., recoding education levels into "High School," "Undergraduate," and "Graduate").
- **Binarization:** Converting a continuous or categorical variable into a binary format (e.g., recoding a variable based on a threshold, such as age > 18 being recoded as "Adult").
- **Creating Ranges or Bins:** Dividing a continuous variable into a set of discrete categories or intervals (e.g., converting age into age groups like "Young," "Middle-aged," "Senior").

R functions for recoding

Function	Purpose	Example
ifelse()	Simple conditional recoding	Replace age values above 30 with "Above 30"
case_when()	Multiple conditions (requires dplyr)	Recode age into "Young", "Adult", and "Senior"
factor()	Recode categorical variables and factor levels	Recode gender from 1, 2 to "Male", "Female"
replace()	Replace specific values	Replace 0 with NA
cut()	Recodes continuous variables into categories	Divide income into "Low", "Medium", and "High" groups
within()	Recoding variables inside a data frame	Create a new variable age_category within a data frame



Manipulating Data in R

- Manipulating data in R involves transforming and organizing your data to suit your analysis needs.
- This process often includes identifying the need for reshaping, filtering, grouping, summarizing, and joining datasets. R's robust data structures and libraries enable efficient execution of these tasks.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



- **Key Data Manipulation Techniques:**
- **Reshaping Data:**
 - **Pivoting:** Converting data between wide and long formats.
 - **Stacking and Unstacking:** Rearranging hierarchical data structures.
 - **Melting and Casting:** Transforming data between different formats.
- **Filtering Data:**
 - **Subset:** Selecting specific rows or columns based on conditions.
 - **Filtering:** Removing rows or columns that don't meet certain criteria.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



- **Grouping Data:**

- **Grouping:** Combining rows based on common values in a specific column.
- **Summarizing:** Calculating aggregate statistics for each group.

- **Joining Data:**

- **Merging:** Combining datasets based on common columns.
- **Concatenating:** Appending datasets row-wise or column-wise.

- **Transforming Data:**

- **Creating New Variables:** Deriving new columns from existing ones.
- **Modifying Existing Variables:** Changing the values of existing columns.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



R functions to Reshape and merge Datasets

Function	Package	Description
<code>pivot_wider()</code>	dplyr	Converts data from long to wide format.
<code>pivot_longer()</code>	dplyr	Converts data from wide to long format.
<code>melt()</code>	reshape2	Converts data from wide to long format.
<code>cast()</code>	reshape2	Converts data from long to wide format.
<code>join()</code>	dplyr	Combines datasets based on common columns.
<code>merge()</code>	Base R	Combines datasets based on common columns.
<code>rbind()</code>	Base R	Combines datasets row-wise.
<code>cbind()</code>	Base R	Combines datasets column-wise.



R Functions for Reshaping and Merging Datasets

Function	Description	Example
reshape()	Reshapes data between wide and long formats.	<pre>df_long <- reshape(df_wide, idvar = "id", timevar = "time", direction = "long")</pre>
merge()	Joins datasets based on common columns.	<pre>merged_df <- merge(df1, df2, by = "id")</pre>
rbind()	Combines datasets row-wise.	<pre>combined_df <- rbind(df1, df2)</pre>
cbind()	Combines datasets column-wise.	<pre>combined_df <- cbind(df1, df2)</pre>



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Code snippet for reshape and merge function

```
library(reshape)
```

```
# Sample data
```

```
df_wide <- data.frame( id = c(1, 2),  
  time1 = c(10, 8), time2 = c(12, 9) )
```

```
# Reshaping from wide to long format
```

```
df_long <- reshape(df_wide, idvar = "id",  
  timevar = "time", direction = "long")
```

cont

Merging datasets

```
df2 <- data.frame( id = c(1, 2),  
                   category = c("A", "B"))
```

```
merged_df <- merge(df_long, df2, by = "id")
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Data Visualization

- **R is a powerful tool for data visualization, offering a wide range of packages and functions to create informative and visually appealing plots.**



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Key Considerations for Data Visualization:

- **Clarity:** Ensure that the plot is easy to understand and interpret.
- **Relevance:** Choose visualizations that effectively convey the message you want to communicate.
- **Aesthetics:** Use appropriate colors, fonts, and themes to enhance the visual appeal of the plot.
- **Interactivity:** Consider using interactive plots to allow users to explore the data more deeply.
- **Accessibility:** Make sure your visualizations are accessible to people with disabilities.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Here are some of the most popular R packages for data visualization:

Core R Graphics

- `plot()`: A versatile function for creating basic plots like scatter plots, line plots, and histograms.
- `barplot()`: Creates bar charts.
- `pie()`: Creates pie charts.
- `boxplot()`: Creates box plots.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Specialized Packages

- **ggplot2**: A grammar of graphics approach, providing a flexible and customizable framework for creating various types of plots.
- **plotly**: Creates interactive plots that can be explored and zoomed in on.
- **ggvis**: An interactive grammar of graphics package.
- **lattice**: A package for creating trellis graphics, which are useful for visualizing relationships between multiple variables.
- **cartogram**: Creates cartograms, which are maps where the size of geographic areas is distorted to represent a specific variable.
- **leaflet**: Creates interactive maps.



Plotting with ggplot2

- ggplot2 is a powerful and versatile package in R for creating a wide range of static and interactive visualizations.
- It follows a grammar of graphics approach, allowing you to build plots layer by layer, providing great flexibility and customization.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



• Basic Structure of a ggplot2 Plot

A typical ggplot2 plot consists of the following components:

- Data: The dataset you want to visualize.
- Aesthetic Mappings: Linking variables in your data to visual properties like color, size, shape, and alpha.
- Geometries: The visual elements used to represent your data (e.g., points, lines, bars).
- Facets: Dividing the plot into subplots based on a variable
- Scales: Controlling the appearance of axes, legends, and other plot elements.
- Coordinates: Transforming the coordinate system (e.g., log scale, polar coordinates).
- Theme: Customizing the overall appearance of the plot.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Key ggplot2 Functions:

- `ggplot()`: Creates a basic ggplot object.
- `geom_*()`: Adds geometric objects to the plot (e.g., `geom_point()`, `geom_line()`, `geom_bar()`).
- `aes()`: Specifies aesthetic mappings.
- `labs()`: Sets labels for the plot title, axes, and legend.
- `scale()`: Controls the appearance of scales (e.g., `scale_x_continuous()`, `scale_color_manual()`).
- `facet()`: Creates facets (e.g., `facet_wrap()`, `facet_grid()`).
- `theme()`: Customizes the overall appearance of the plot.



Code snippet

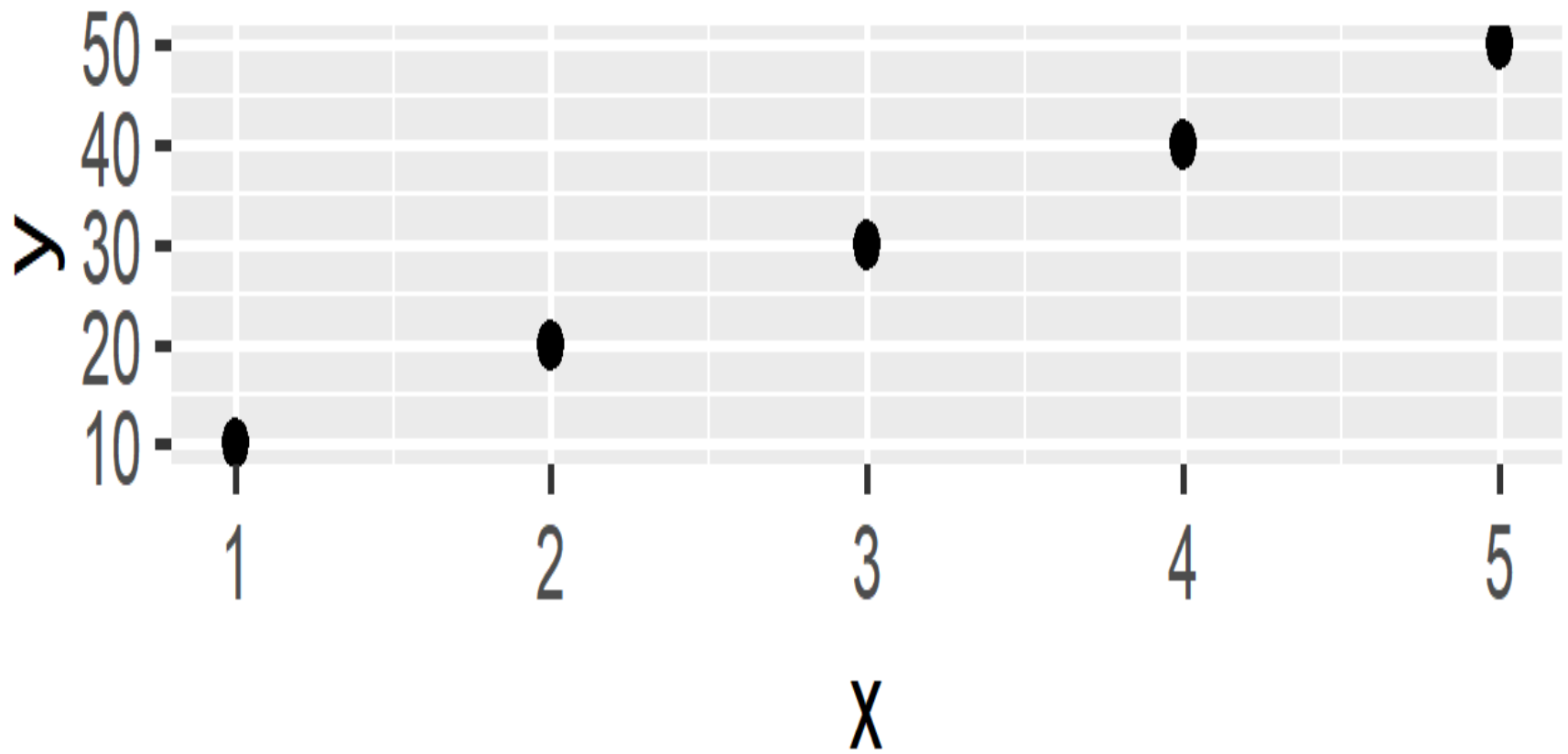
```
library(ggplot2)
x <- c(1, 2, 3, 4, 5)
y <- c(10, 20, 30, 40, 50)
df <- data.frame(x, y)
ggplot(data = df) + geom_point(aes(x = x, y = y))
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013





**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Creating Simple Plots with ggplot2

- **Scatter plot**

```
library(ggplot2)
```

```
# Sample data
```

```
df <- data.frame( x = 1:10, y = rnorm(10))
```

```
#Create a scatter plot
```

```
ggplot(df, aes(x, y)) + geom_point() + labs(title =  
"Scatter Plot", x = "X-axis", y = "Y-axis")
```

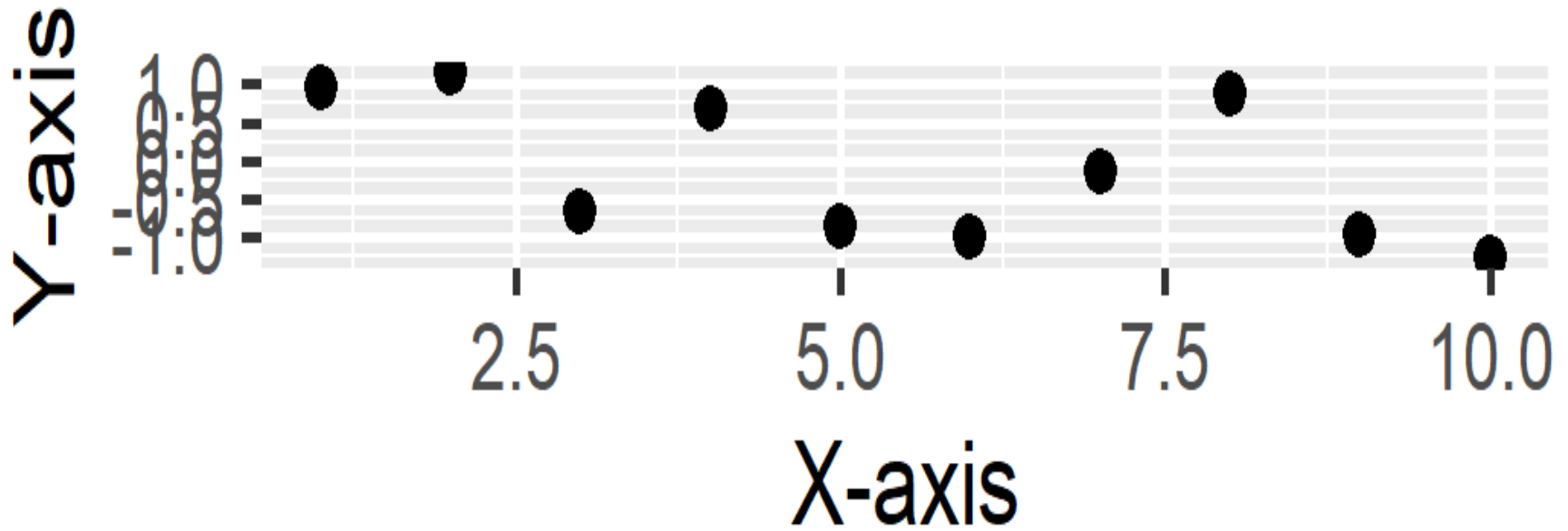


**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Scatter Plot

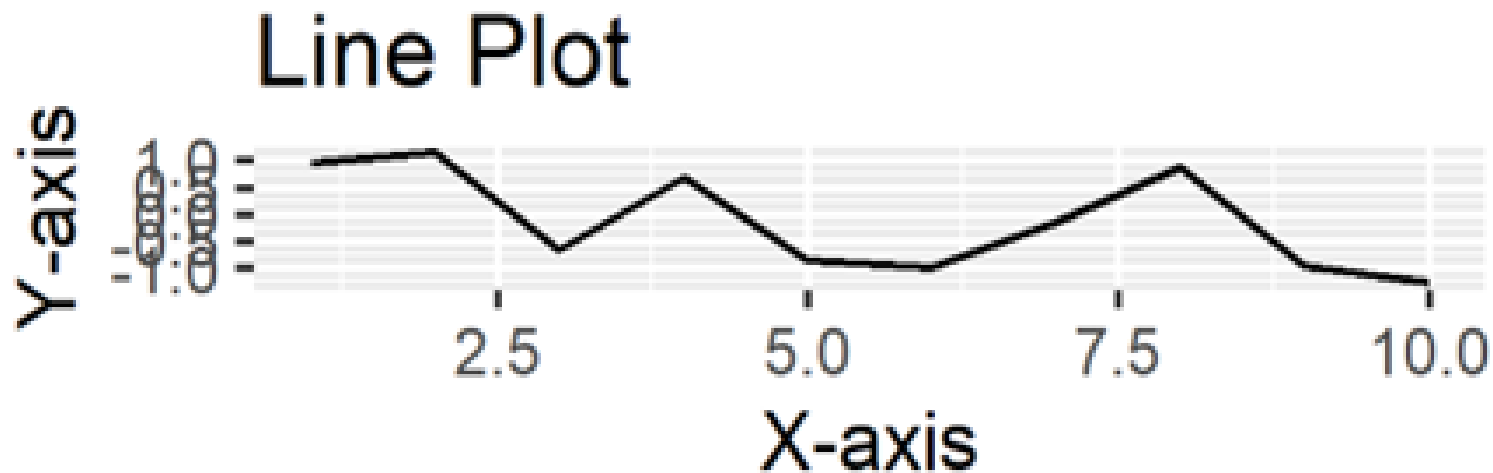


**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



- # Create a line plot
- `ggplot(df, aes(x, y)) + geom_line() + labs(title = "Line Plot", x = "X-axis", y = "Y-axis")`



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Create a bar plot

```
df_bar <- data.frame(  
  category = c("A", "B", "C"),  
  value = c(10, 15, 20)  
)  
  
ggplot(df_bar, aes(category, value)) +  
  geom_bar(stat = "identity") +  
  labs(title = "Bar Plot", x = "Category", y = "Value")
```

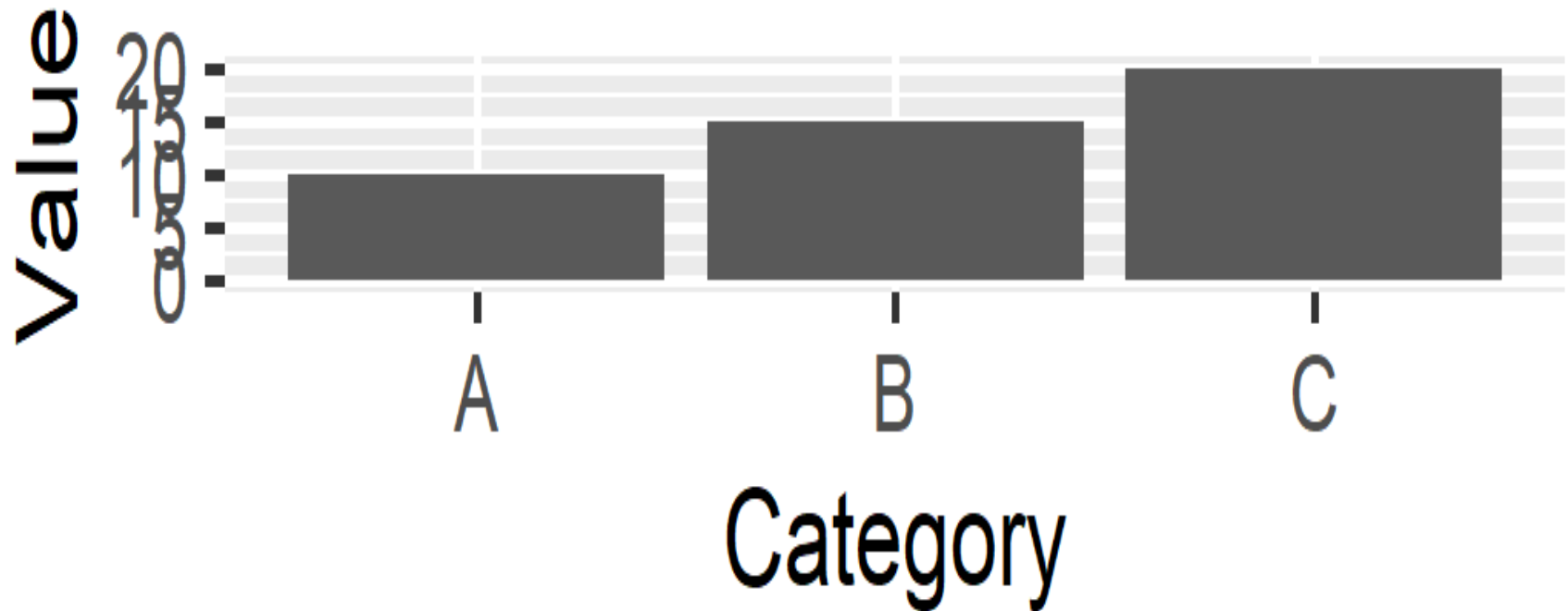


**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Bar Plot



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013

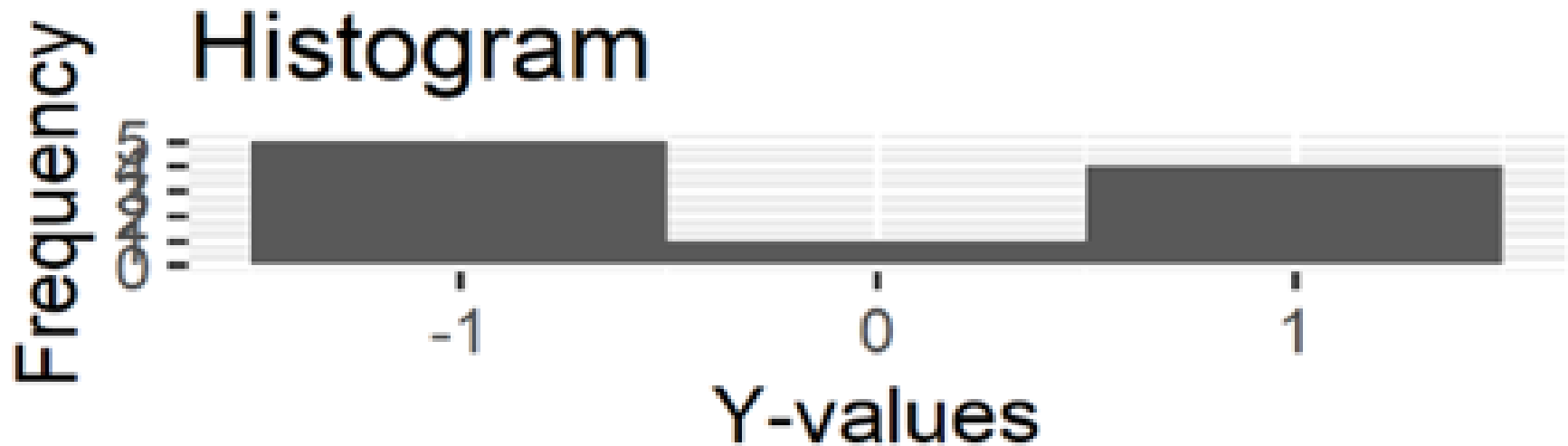


- # Create a histogram

```
ggplot(df, aes(y)) +
```

```
geom_histogram(binwidth = 1) +
```

```
labs(title = "Histogram", x = "Y-values", y =  
"Frequency")
```



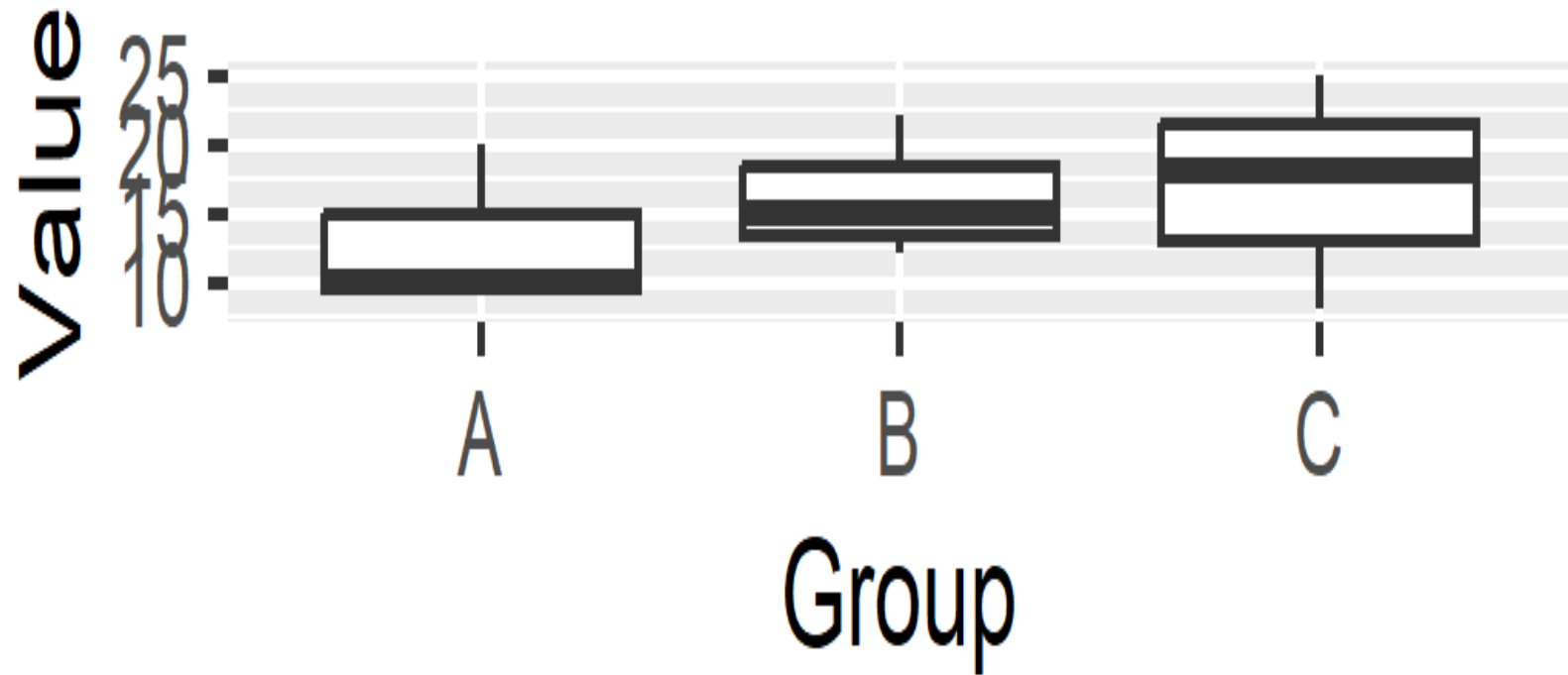
**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



```
# Create a box plot
df_box <- data.frame(
  group = c("A", "B", "C"),
  value = c(10, 12, 8, 9, 15, 18, 20, 22, 25)
)
ggplot(df_box, aes(group, value)) +
  geom_boxplot() +
  labs(title = "Box Plot", x = "Group", y = "Value")
```

Box Plot



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Plotting with Base R

While ggplot2 is a popular choice for data visualization in R, base R also offers a wide range of plotting functions that can be used to create various types of plots.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



- **Basic Plotting Functions:**

- **plot():** A versatile function for creating scatter plots, line plots, and histograms.
- **barplot():** Creates bar charts.
- **pie():** Creates pie charts.
- **hist():** Creates histograms.
- **boxplot():** Creates box plots.



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Additional Plotting Functions:

- **pairs()**: Creates scatter plots for all pairs of variables in a data frame.
- **matplot()**: Creates multiple line plots on a single graph.
- **image()**: Creates heatmaps.
- **contour()**: Creates contour plots.

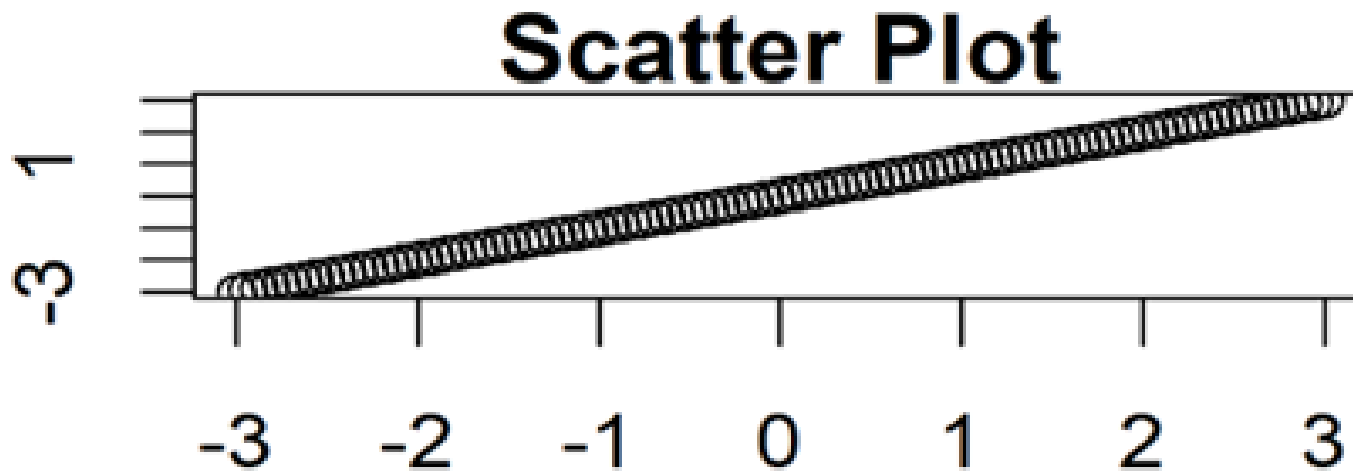
Creating Basic Plots with Base R Functions

- Scatter Plot

```
par(mar = c(2, 2, 1, 1))
```

```
# Create a scatter plot
```

```
Plot(x, y, main = "Scatter Plot", xlab = "X-axis", ylab = "Y-axis")
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



#Line plot

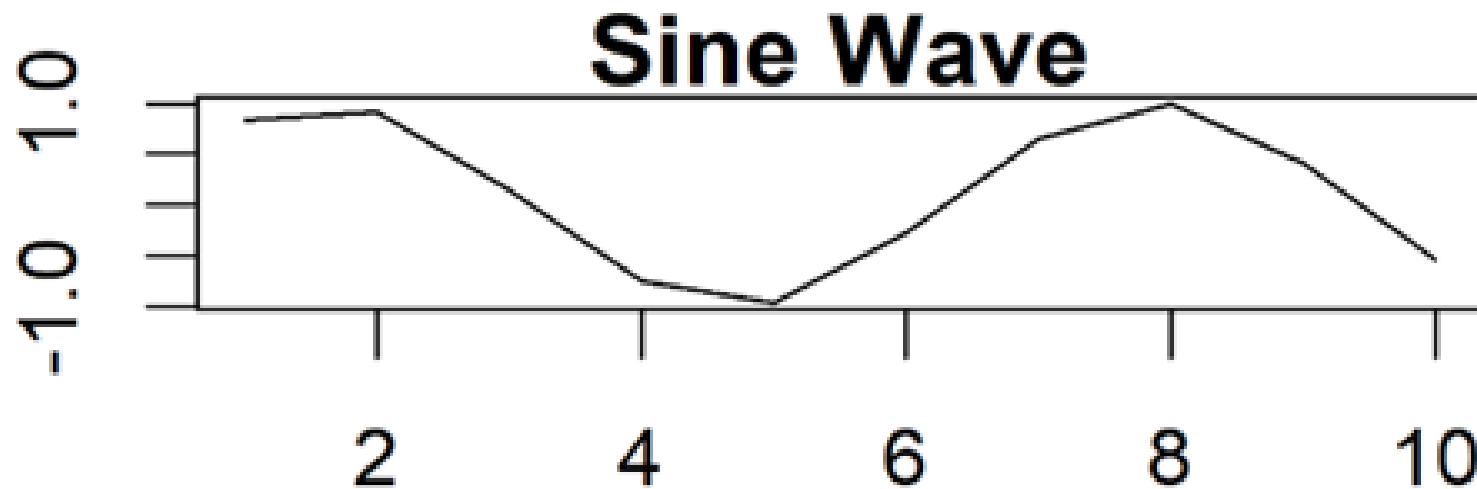
```
par(mar = c(2, 2, 1, 1))
```

```
# Create a line plot
```

```
x <- 1:10
```

```
y <- sin(x)
```

```
plot(x, y, type = "l", main = "Sine Wave", xlab = "X-axis", ylab = "Y-axis")
```



**PRESIDENCY
UNIVERSITY**

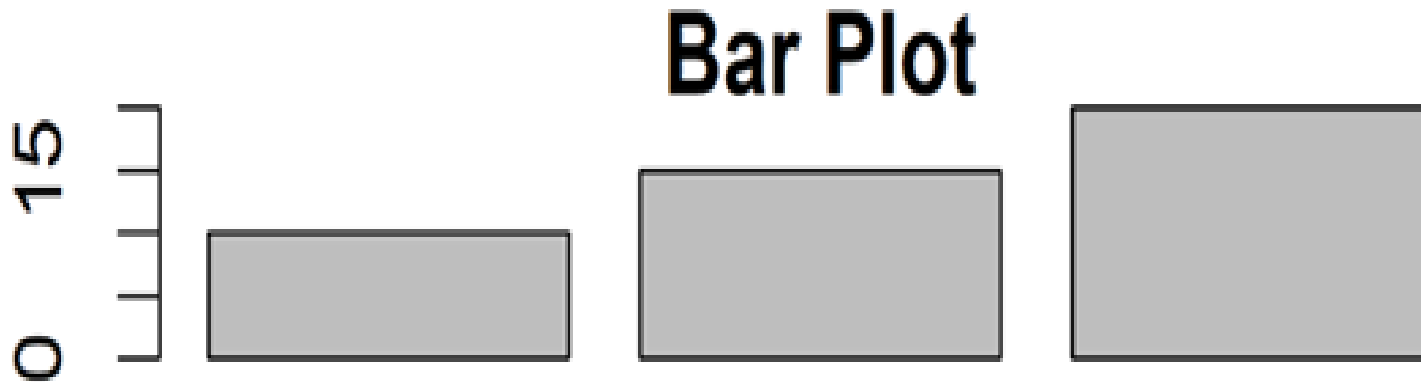
Private University Estd. in Karnataka State by Act No. 41 of 2013



#Bar plot

```
data <- c(10, 15, 20)
```

```
barplot(data, main = "Bar Plot", xlab =  
"Categories", ylab = "Values")
```



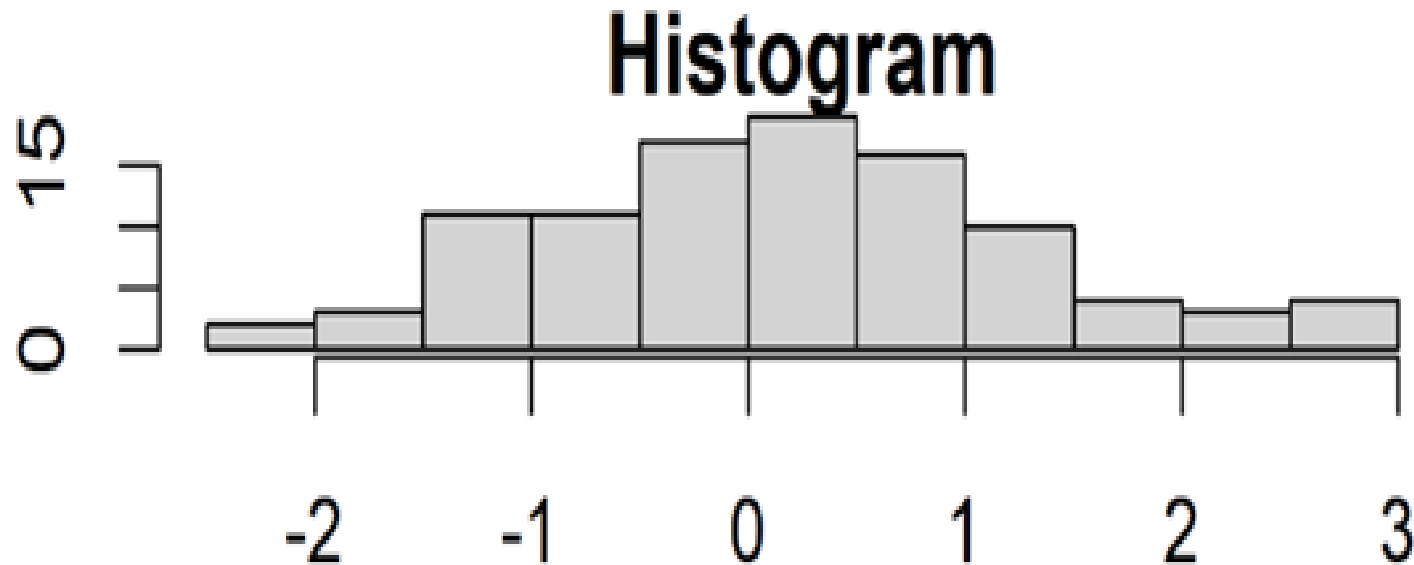
**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



#histogram

```
hist(rnorm(100), main = "Histogram", xlab =  
"Values", ylab = "Frequency")
```



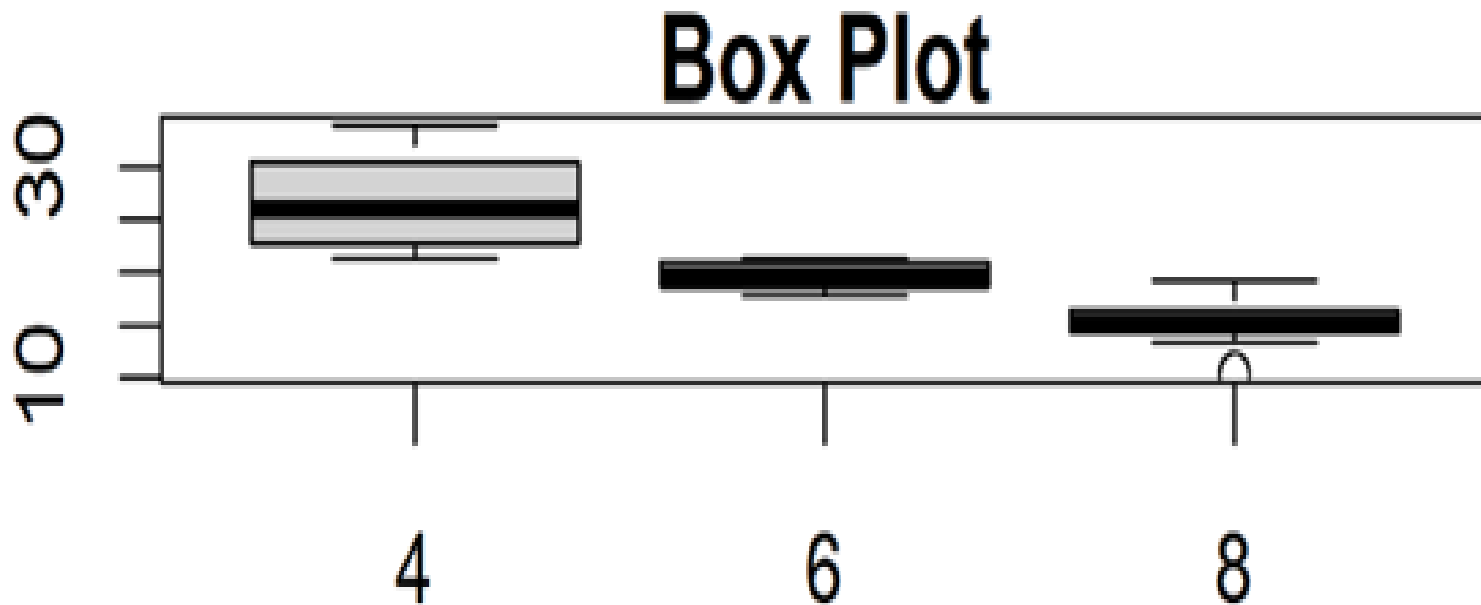
**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



#boxplot

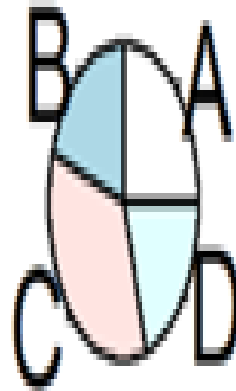
```
boxplot(split(mtcars$mpg, mtcars$cyl), main =  
"Box Plot", xlab = "Cylinders", ylab = "Miles Per  
Gallon")
```



#pie

```
pie(c(25, 20, 35, 20), labels = c("A", "B", "C", "D"),  
main = "Pie Chart")
```

Pie Chart



#contour

```
x <- seq(-3, 3, length.out = 100)
```

```
y <- seq(-3, 3, length.out = 100)
```

```
z <- outer(x, y, function(x, y) x^2 + y^2)
```

```
contour(x, y, z, main = "Contour Plot")
```

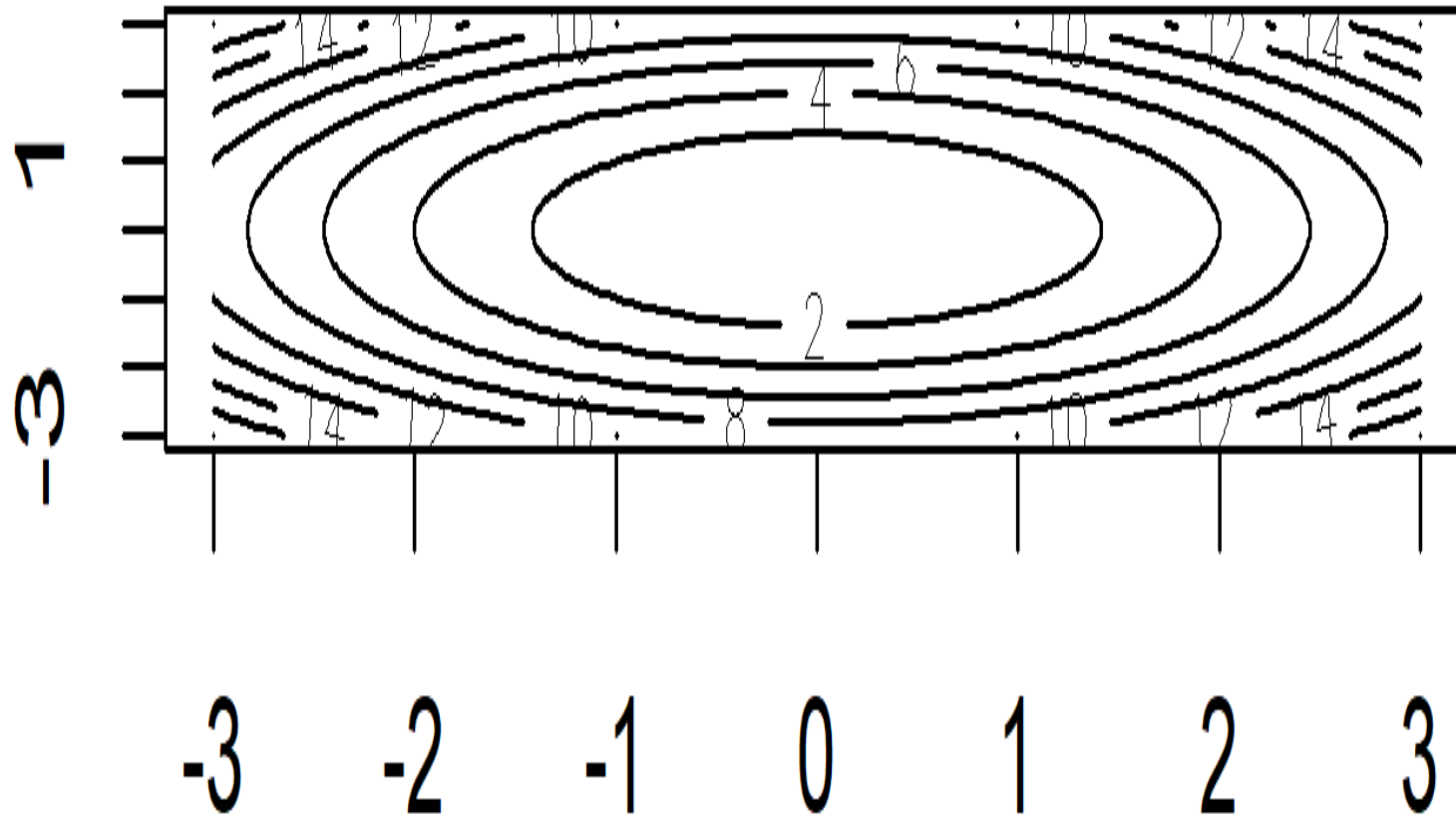


**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



Contour Plot



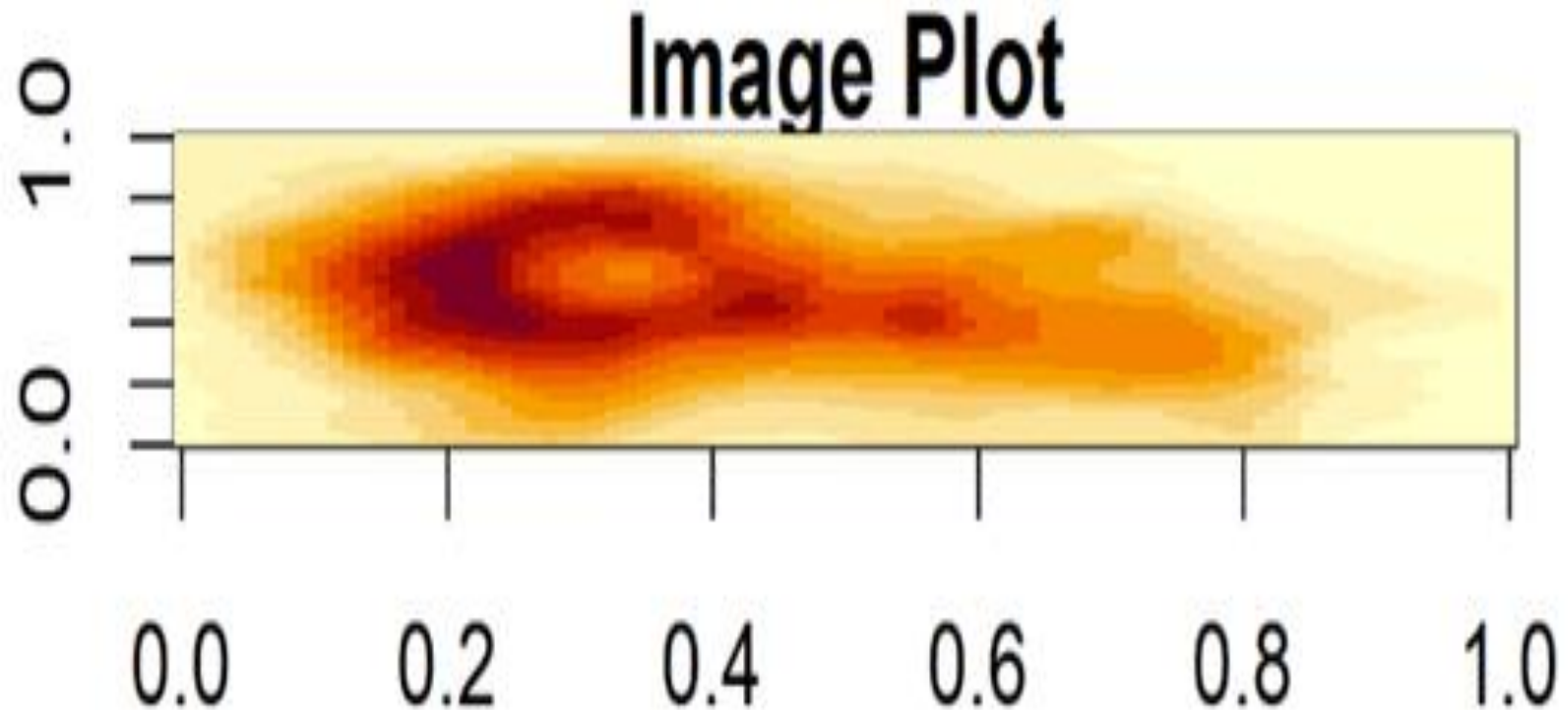
**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



#imageplot

`image(volcano, main = "Image Plot")`



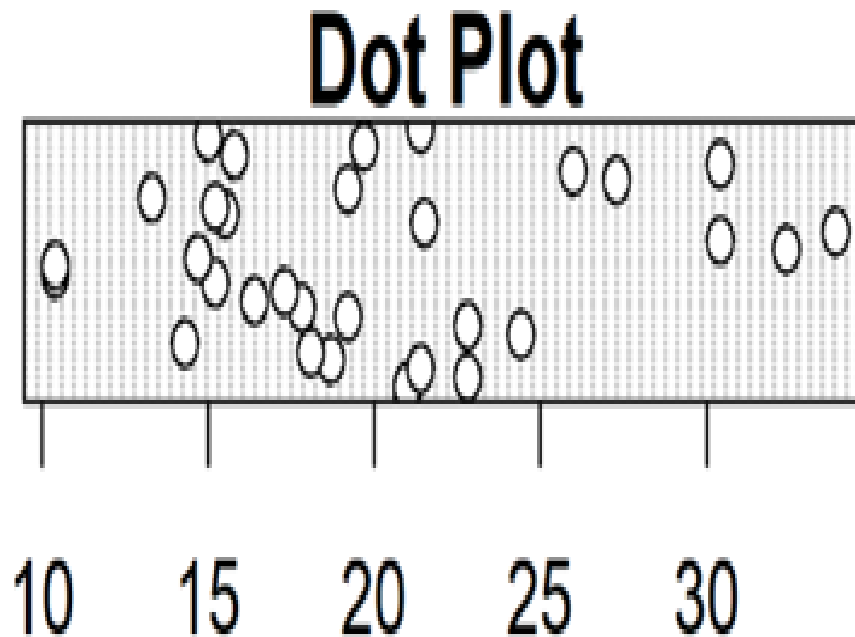
**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013



#dot

```
dotchart(mtcars$mpg, labels =  
rownames(mtcars), main = "Dot Plot")
```



**PRESIDENCY
UNIVERSITY**

Private University Estd. in Karnataka State by Act No. 41 of 2013

